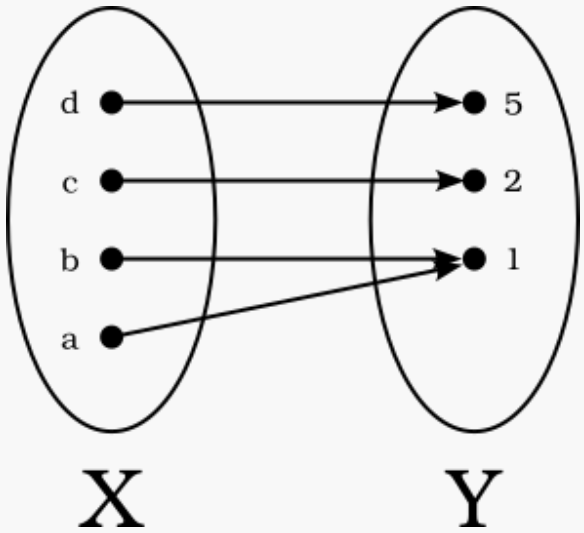
11 - Maps

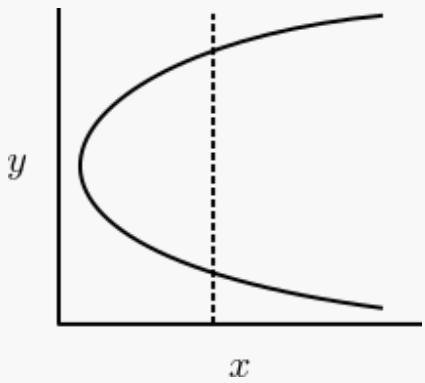
A map in terms of formal math is described as a "surjective onto relationship", but what does this mean practically and where have we seen maps in the real world? In fact, we rely on maps frequently, especially in an online world, and at university, one map is critically important to student life.



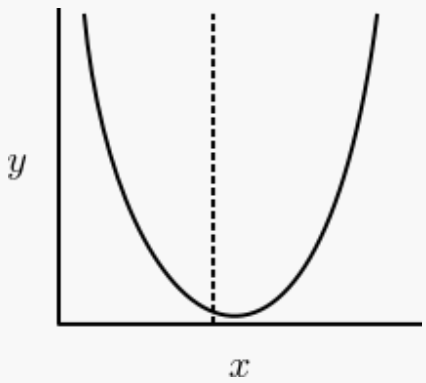
Mapping X onto Y

A mathematical function is defined as $f : X \to Y$ where X is the set (domain) of all input values and Y is the set (co-domain) of all associated output values. This can be interpreted as, f maps the values in X onto the values of Y and we often use the notation $f(x)$ to clearly represent that x is an input and $f(x)$ is the mapping. In other words, a mathematical function is one form of map. What are the key characteristics of a function that define it as a map?

Recall that a simple, graphical way to determine whether or not a plot is a function is that the plot must pass the vertical line test in order to be a function. The **vertical line test** is defined as “if a vertical line can be drawn anywhere on a graph such that the vertical line intersects the plot of a function at more than one point, then the plot can not be a function”.



Fails the Vertical Line Test



Passes the Vertical Line Test

In the above example, the left plot is not a function because it is trivial to draw a vertical line that intersects the plot in multiple locations, and the right plot is a function (at least the portion that can be seen in the graph) because no matter where we draw vertical lines, every vertical line would only intersect the plot in a single location. The vertical line test may be an easy tool to use with a plot; however, what does the vertical line test actually test and how can we determine whether or not an arbitrary set of input and output data meets the same requirements?

The vertical line test validates that for every *unique* value of x there exists an associated value of y. In other words, each value in the X domain maps to one and only one value in the Y. Note that this is a one-way relationship and does not limit values in the Y domain. Two different x's can map to the same y value but two y values cannot map to the same x value. A function is therefore only defined by the mapping of X onto Y using the vertical line test and there is no corresponding horizontal line test. In terms of relationships, we can say there is a **one-to-one relationship** between x's and y's, *i.e.* for each x there exists one and only one associated y value, and we can say there is a **one-to-many relationship** between y's and x's, *i.e.* for each y there may exists may associated x values.

So where do we encounter maps beyond just mathematical functions? Believe it or not, maps run the world and you rely on them with just about every interaction on the Internet. One form of map is the email address you used to sign up for an account, another map is the underlying IP address assigned to your device's Internet connection, and yet another map is your GWID or Social Security Number. In each of these cases, your information is associated with a single, unique key that serves as an identifier for you.

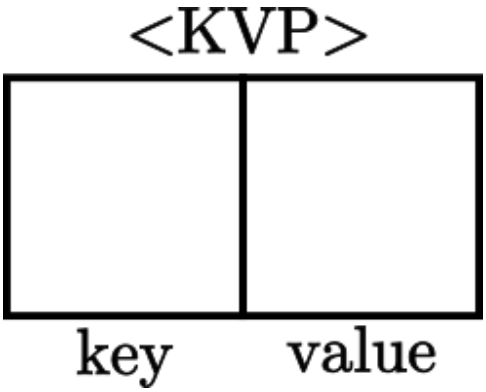
key	**value**		
gwid	first_name	last_name	gwmail

1	gwid	first_name	last_name	gwmail
2	G12322520	Cristie	Bricknell	cbricknell0@gwu.edu
3	G58213704	Rodi	Axtell	raxtell1@gwu.edu
4	G64041832	Marianna	Shorter	mshorter2@gwu.edu
5	G96159828	Candis	Bowen	cbowen3@gwu.edu
6	G37089253	Cindra	Dauby	cdauby4@gwu.edu
7	G04101191	Zabrina	Kimm	zkimm5@gwu.edu
8	G61875038	Stefania	Bomfield	sbomfield6@gwu.edu
9	G71301543	Amabelle	Frape	afrape7@gwu.edu
10	G32892599	Maitilde	Rome	mrome8@gwu.edu
11	G11306026	Fayre	McCrostie	fmccrostie9@gwu.edu
12	G32673844	Abeu	Klemensiewic	aklemensiewicza@gwu.edu
13	G04771723	Elwyn	Binford	ebinfordb@gwu.edu
14	G96262557	Barnaby	Wince	bwincec@gwu.edu
15	G18428793	Hansiain	Mantram	hmantramd@gwu.edu
16	G19223000	Nadia	Mosten	nmostene@gwu.edu
17	G29276578	Moise	Paulton	mpaultonf@gwu.edu
18	G07057814	Donnie	Otson	dotsong@gwu.edu
19	G33605956	Doralynn	Sargison	dsargisonh@gwu.edu
20	G67761933	Silvanus	Caw	scawi@gwu.edu

In a database, the key acts as a unique identifier for a specific set of data or record and it is a necessary feature for fast lookup of records and to minimize data duplicated.

Key-Value Pair

In programming, we express and maintain map associations by storing **key-value pair (KVP)** data



Regardless of the specifics of a particular implementation, any map must maintain the association of key to its value and it is typically managed by a structure we call the **Key-Value Pair** or **KVP** for short. It can be simply represented as a tuple consisting of two elements (very common in python), an array of 2 elements, or in OOP languages as an explicit class with accessors and mutators that limit the ability to change keys after the object has been initialized.

In addition to being unique, the key is generally considered **immutable** meaning that it cannot be changed once it has been created. From a practical development standpoint, this probably means that the key is a read-only field in the KVP structure while it is allowable to both read and write to the value field, so the key should have an accessor only in the KVP class interface while the value may have both an accessor and a mutator in the KVP class interface.

Note that as suggested above in the example student data, a value can be rather complex and can be composed of a number of properties all of which are changeable.

Map ADT Interface

There are three basic operations needed in a map all of which must test for the existence of the key in the map and their behaviors are dependent on the outcome of this test.

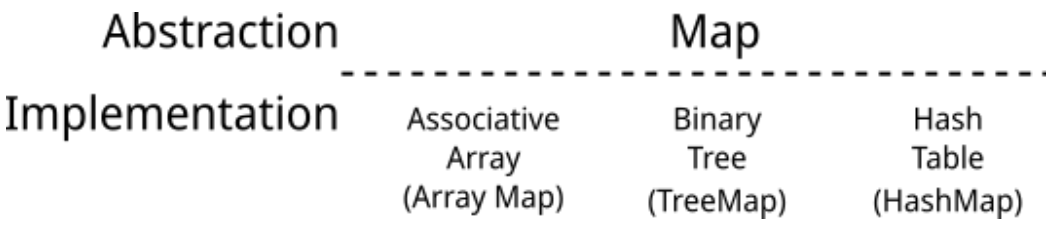
```
put( key, value ) : if the key exists in the map, put updates that key with the associated
value; otherwise, put inserts the key-value pair into the map

value get( key ) : if the key exists in the map, get retrieves the associated value; otherwise,
get returns a signifier that the key does not exist in the map

value delete( key ) : if the key exists in the map, delete removes the key and the associated
value; otherwise, delete returns a signifier that key does not exist in the map
```

It is important to note that the above public interface is built on a number of common, basic operations hidden in the implementations including search, update, and insert. Due to the uniqueness constraint for the keys that are used by maps, `put` must always search for the key first. It is only after the search is complete that we know whether `put` is performing an update or an insert. If the search fails to find the key in the map, then `put` performs an insert at the node where the search ended, and if search successfully finds the key, then `put` performs an update at the node where the search ended. Similarly, in order to `get` or `delete`, a search needs to be performed first to locate the key in the map and if the search fails then these operations must fail gracefully, that is fail in such a way that does not cause the program to halt.

To **insert** into a map is to add the KVP into the structure in a way that maintains any constraints such as order or balance that is critical to maintaining the expected search efficiency of the structure. To **update** a map is to change the associated value to a new value for an existing, specified key without otherwise altering the data structure.

Implementations

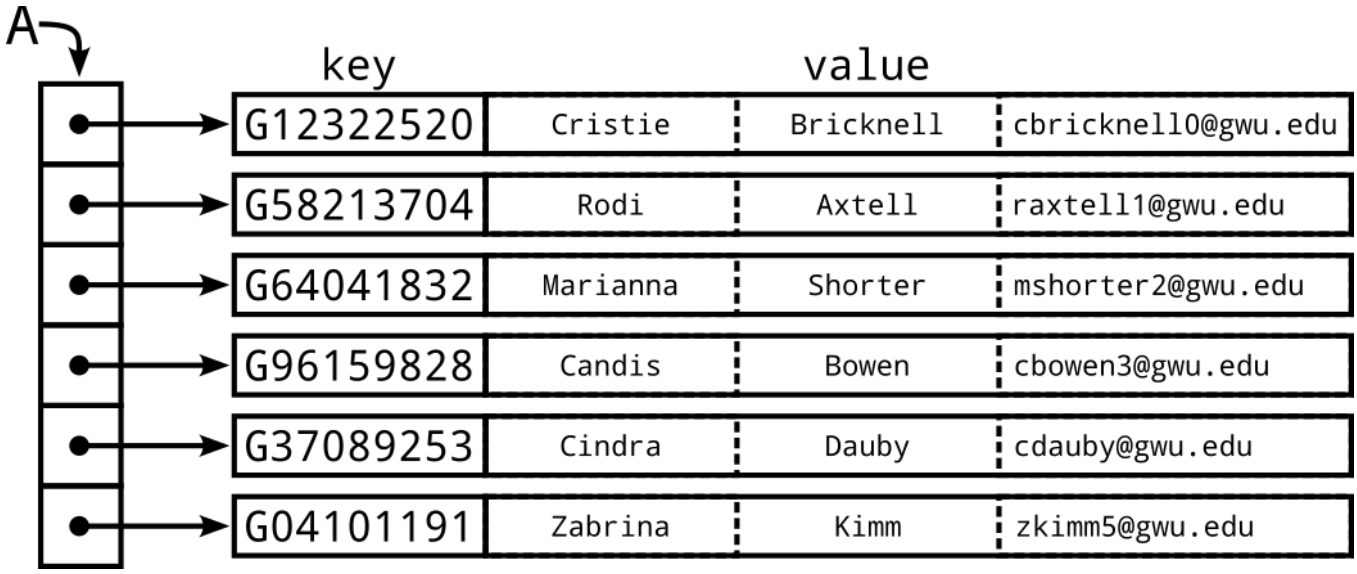
As with any ADT, there are a variety of ways we can implement a map as long as the map maintains uniqueness of keys, the key-value association, fulfills the Map interface, and consistently maintains its search efficiency without degenerating.



Regardless of the implementation, since the key in the key-value pair is an alias for the input set of domain X, **any individual key must be unique in the set of all keys**, so whenever a key-value pair is inserted into a map, the map must always enforce the uniqueness of the key and there can never be allowed a duplicate of the key in the map. Therefore it is the primary responsibility of the map implementation to enforce uniqueness of the key for all operations associated with the map, and in particular for any write operations in the map, *i.e.* `put` and `delete`. This means that interactions must first search for the existence of the key as a part of their behaviors and their overall efficiency is subject to the efficiency of search within the data structure. Efficient search is therefore a top priority of any map implementation.

Associative Array

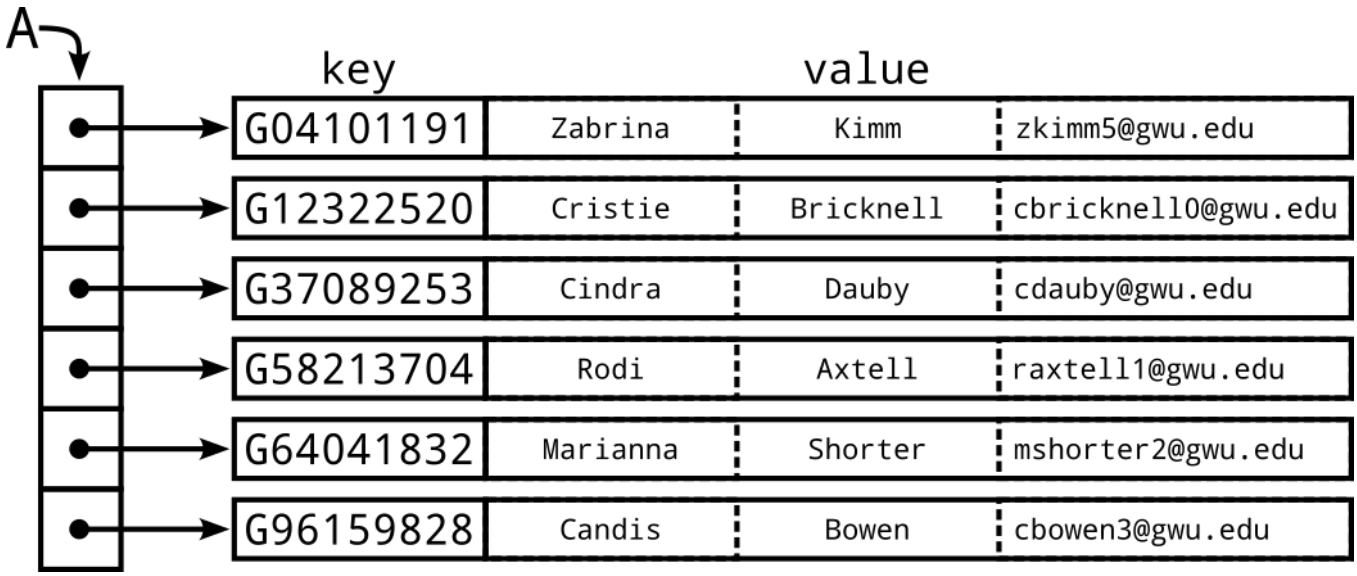
As with most ADTs, we can use an array to implement a map and in this context we will call this type of array an associative array or array map. Consider a subset of the student data presented before.



The data could be represented as a set of records ordered or otherwise using an array. Consider the efficiency of `get`, `put`, and `delete` in the associative array.

If the array is unordered, then the efficiencies of `put`, `get`, and `delete` are all $O(n)$ because we have to first linear search for any map operation because we must first determine whether or not the key exists in the data structure before fulfilling the operation. The efficiency of linear search in an unordered array is $O(n)$ where n is the number of keys in the array, so a map using an unordered array will generally be an $O(n)$ data structure for all map operations. For a proof of the efficiency of linear search in an array refer to *Chapter 5 - Search*.

However, we could instead maintain the array as an ordered set ordered on the key, in this case the student identifier, allowing us to use binary search on the array instead of linear search.



if the array remains ordered by the key, then the binary search worst case efficiency of $O(\log_2 n)$ where n is the number of keys in the array determines the overall efficiency of the data structure in terms of search. Since `get(key)` is just a search, then the efficiency of `get(key)` will be $O(\log_2 n)$ as long as the array is maintained in order on the key field. For a proof of the efficiency of binary search in an array refer to *Chapter 5 - Search*.

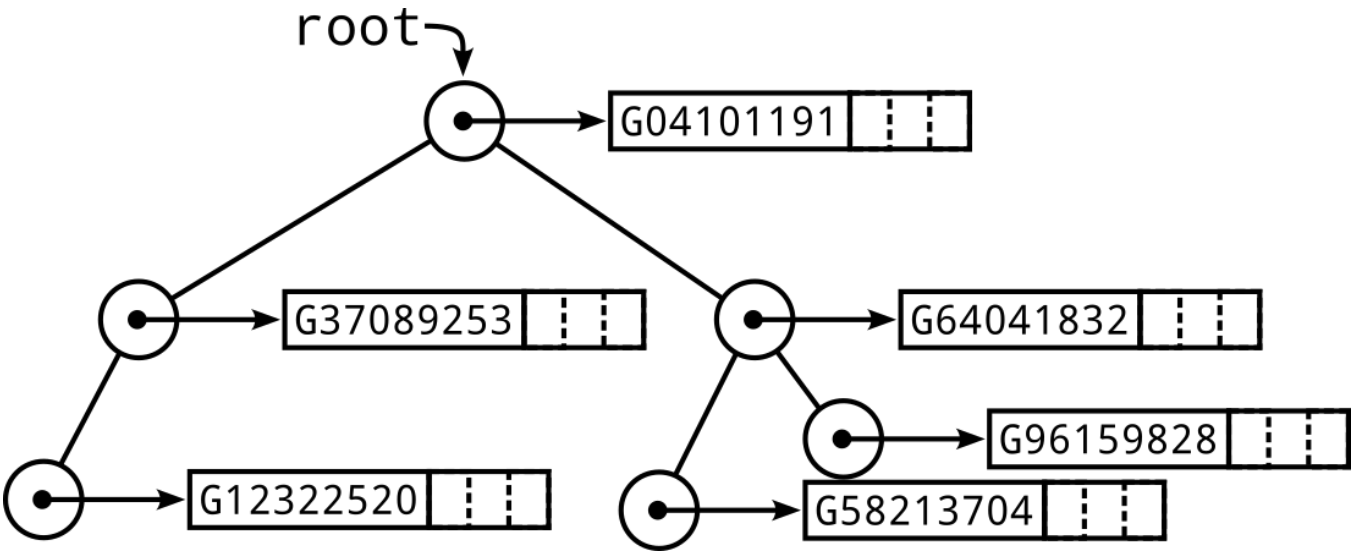
Recall that `put(key,value)` first searches the array for the existence of the key and if the key is found, the value is updated; otherwise, the key-value pair is inserted into the array. Since the array is in order, we can use binary search to answer the existential question; however, our worst-case for this 'search and insert' operation actually emerges from maintaining order when inserting or removing an element and not from the search. If the key does not exist, we must insert the key-value pair into an ordered array, so we would need to make space for the new element. Regardless of whether or not spare empty space is preallocated in the array, we would still need to shift all elements over to insert the element into its correct ordered position. If the element already exists in the array, then `put` will just update the value and the shift is not needed so the update is not a worst-case operation. Therefore in the worst-case, `put` in the associative array is $O(n)$ where n is the number of keys in the array.

`delete(key)` also suffers from the same efficiency issue just in reverse. Even though we don't perform an insert *per se*, the process for `delete` is effectively the same except for the final insertion step from worst case `put`, *i.e.* we first search and then we shift skipping the final insertion. For `delete`, we need to shift elements back toward the low-order indices in order to fill the space left by the element we removed as we do not want to have gaps in the array.

Using an associative array is feasible, but for many operations other than `get` in an ordered array, the efficiency is generally no better than $O(n)$. This begs the question of whether or not there exists a data structure we can implement for maps that will give us consistently better efficiency than the associative array.

Binary Search Tree

One alternative might be to store the key-value pairs in a binary search tree and search in the keys, raising the question of whether or not the characteristics of a BST match well to a map. A BST by its very definition is ordered, its search efficiency is $O(\log_2 n)$ as long as it remains balanced, and it is possible to rebalance a tree in $O(\log_2 n)$ time. Given these characteristics, it suggests that most if not all operations of a map can be supported in $O(\log_2 n)$ time complexity.



The map interaction `get(key)` is simply a search on a key. If a BST is ordered by keys, then the existential question for a key existing in the tree can be determined based on the efficiency of search in a BST and once the key is found, the value can be retrieved from the identified node in constant time. The efficiency of search in a BST is $O(\log_2 n)$ where n is the number of elements in the tree as long as it remains balanced. For a proof of the

efficiency of search in a BST refer to *Chapter 10 - Trees*.

The map interaction `put(key, value)` involves first a search followed by either an update or insert. As established with `get(key)` the efficiency of search in the BST is $O(\log_2 n)$, so the question is what are the efficiencies of update and insert. The process of searching the BST would result in locating either the node in which the key resides or the node to which the key should descend from on an insert. Updating the associated value for a key-value pair should be no worse than constant time, so a `put` that results in update should be $O(\log_2 n) + O(1)$ reducing to $O(\log_2 n)$ in the worst case. Inserting after already locating the parent node of the key to insert should also be no worse than constant time followed by a potential rebalance along the search path. Recall that a failed search in a BST means that we end at a node that does not have a child in the expected direction where the key we are searching on should lie, so adding a new KVP at this position requires only allocating a new node, attaching that new node to the unassigned branch of the node reached during search making the insertion part of `put` a constant time operation. If rebalancing is $O(\log_2 n)$, a `put` that results in insert should be $O(\log_2 n) + O(1) + O(\log_2 n)$ or overall $O(\log_2 n)$ in the worst case. Therefore in a BST, the efficiency of `put` is $O(\log_2 n)$ as long as it remains a properly implemented BST.

The map interaction `delete(key)` involves first a search followed by a rebalancing of the tree along the search path. To efficiently delete, we can mark a node for deletion and then rebalance that node which will ideally cost no more to rebalance the subtrees associated with that node than the normal rebalancing operation. If this methodology works, then `delete` is no worse than a search followed by a rebalancing operation which would again be $O(\log_2 n) + O(1) + O(\log_2 n)$ or $O(\log_2 n)$ in the worst case. Therefore in a properly implemented BST, the efficiency of `delete` is $O(\log_2 n)$.

Clearly the BST is a better candidate in terms of efficiency than the associative array. This does not mean we can't use or should never consider the associative array. There are some applications where efficiency may not be our top concern and the associative array may be functionally the easiest and most reliable choice; however, if efficiency is a high priority, then we should choose the BST over the associative array. But, is there something even more efficient than a BST?

Review

1. The term "map" derives from mathematics. Explain symbolically the mapping of one domain onto another. What are the requirements for each domain?
2. Why is the map a one way relationship? What does it mean for values in domain X to have a one-to-one relationship with values in domain Y while values in domain Y have a many-to-one relationship with values in domain X?
3. What is the vertical line test with respect to the definition of a mathematical function and how does this relate to the idea of a map?
4. In computing, we map what to what? In other words, we typically develop a class to handle the mapping association. What is the typical definition for this class, *i.e.* what is a typical name for this class and what fields are defined for this class? In other words, what specific labels do we give to the abstract ideas of domains X and Y from our mathematical map definition?
5. What is the interface for a map, *i.e.* what are a map's prescribed interactions and behaviors?
6. Why must keys be unique?

7. Why does the uniqueness of keys imply all map operations are search?
8. Describe the implementation of an associative array for a map.
9. What is the efficiency of each map interface method for the associative array?
10. Should we insert new elements in order into the associative array or should we periodically sort the array? In other words, what would be the efficiency of inserting into an ordered array versus the efficiency of sorting?
11. Describe the implementation of a binary tree for a map, *i.e.* a treemap.
12. What is the efficiency of each map interface method for the treemap?
13. What is the relationship between the number of levels and the number of elements in a balanced binary tree? How does this relate to the maximum number of steps in search of the binary tree?
14. What is the degenerate case for a tree map? Why might a degenerate case arise in a binary tree? What is the efficiency of the degenerate case? What data structure does the binary tree degenerate into in the degenerate case? Why?